

Docker

Containerizing Services

Our Project

1. Managed with git
2. **Running Docker containers**
3. Orchestrated with Kubernetes
4. Tested in Gitlab
5. Deployed with ArgoCD
6. Monitored with Prometheus and Grafana

Why Containerization?

Modern DevOps projects have many separate components

Each component requires its own libraries and configuration

We wish to deploy, scale, and upgrade each component independently from others

Containerization is one way to allow this

Docker Containers

- Encapsulate application and libraries
- Isolates from rest of system
- Shares kernel with host
- Build on two Linux primitives: **namespaces** and **cgroups**

Docker: Namespaces

Linux namespaces isolate processes in the container from the rest of the system

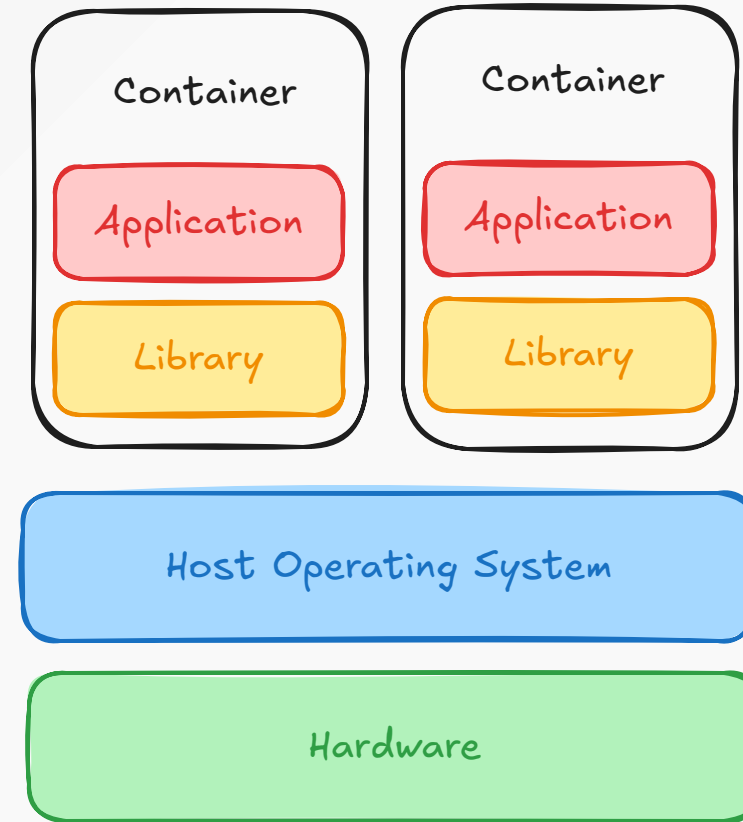
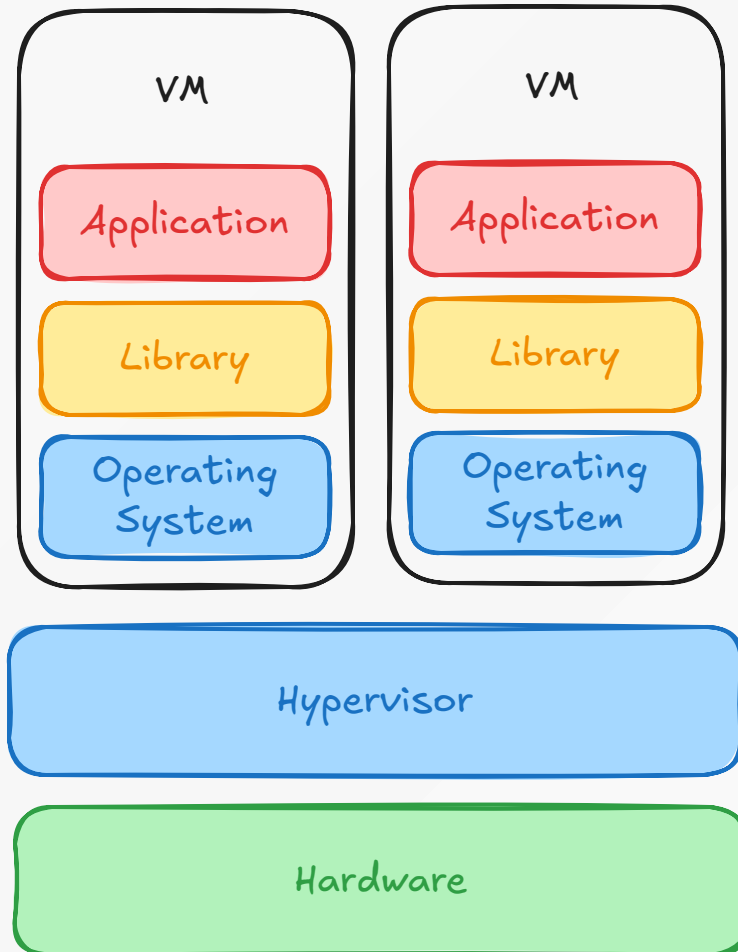
- **Process ID:** Processes in the container cannot see those outside
- **Network:** Container has own network stack, IP, ports
- **Mount:** Isolated file system mounts
- **User:** Separate user and group IDs; container root is not host root
- Others, including **UTS, IPC, cgroup**

Docker: Cgroups

Control groups limit the amount of resources processes in a container can consume:

- **CPU**
- **Memory**
- **Disk I/O**
- **Network bandwidth**

Virtual Machines vs. Containers



Container Escapes

Shared kernel means flaws in isolation can let container processes escape into host

- CVE-2024-21626 ("Leaky Vessels"): Leaked file descriptor in *runc*
- CVE-2025-23266 ("NVIDIAScape"): LD_PRELOAD injection with NVIDIA container toolkit
- CVE-2026-34040: Bypass authentication in Docker Engine
- CVE-2026-53362, CVE-2026-46242 ("Bad Epoll"), CVE-2026-43499 ("GhostLock"): kernel-level escapes

Docker Elements

Dockerfile: Text file describing how to create an image

Image: Files containing the filesystem for a new container

- Stored as layers, identified by content hash

Container: Running processes, with filesystem from image

- Can be stopped and restarted
- Overlay filesystem to persist changes

Starting a Container

```
docker run -it ubuntu:26.04
```

- `-i` Keep stdin open, so we can interact with the container
- `-t` Allocate a pseudo-TTY, so we get a terminal prompt
- `ubuntu` Name of image, here an official image on hub.docker.com
 - `user/image-name` for other users of Docker Hub
 - `example.com/image-name` for images hosted elsewhere
- `:26.04` Image tag, indicating version
 - Defaults to `:latest`

Tracking Containers

```
docker container list
```

Shows all running containers

→ See stopped containers as well with `-a` argument

Containers are automatically given names

→ Can also specify with `--name` argument to `docker run`

Activity: Named Container

1. Stop the current container (`exit` at command prompt)
2. Create a new container with a memorable name
3. In another terminal, get a list of all containers, both the first container that's stopped and the second one you just started

Accessing Running Containers

Execute commands with a running container with `docker exec`

```
docker exec container-name whoami
```

→ Execute the `whoami` command inside `container-name`

```
docker exec -it container-name bash
```

→ Start an interactive *bash* shell inside `container-name`

Managing Containers

Stop the container:

```
docker stop container-name
```

Restart a stopped container:

```
docker start container-name
```

Delete a container (must be stopped):

```
docker rm container-name
```

Discussion: DevSecOps Principles

How do named containers fit into DevSecOps principles?

1. Repeatability and automation
2. Configuration as code
3. Store configuration in version control
4. Declarative, not procedural
5. Livestock, not pets
6. Testing

DevSecOps Principles and Named Containers

1. ~~Repeatability and automation~~
2. ~~Configuration as code~~
3. ~~Store configuration in version control~~
4. Declarative, not procedural
5. ~~Livestock, not pets~~
6. Testing

Recommendation: Run containers with `--rm` to delete at exit
⇒ Automate any annoying set up

Using Ephemeral Containers

```
docker run -it --rm ubuntu:26.04
```

Container will be deleted when it stops

Any configuration will need to be repeated next time

⇒ Script any configuration steps

⇒ Build custom image already configured

All state stored in container will be lost

⇒ Store important state on docker host; mount onto container

Docker Volumes

Docker **volumes** mount host directory inside container

```
mkdir shared # Create directory to mount
```

Preferred syntax:

```
docker run -it --rm --mount type=bind,source=$(pwd)/shared,target=/mnt/volume \
  ubuntu:26.04
```

- **source=** Absolute path on host (error if does not exist)
- **target=** Absolute path in container

Docker Volumes

Legacy syntax:

```
docker run -it --rm -v $(pwd)/shared:/mnt/volume ubuntu:26.04
```

- `-v /path/on/host:/path/in/container`
- Creates host path, if doesn't exist
- If host path not absolute, treated as volume name
 - Docker manages storage

Example: Python Web Server

See `project/images/web-service`

- Python web server, implemented in Flask
- Basic visit counter
- LLM chat interface
- Environment managed by *uv*

We don't have these libraries installed in the Codespace

Digression: Lock files

Project dependencies stored in two files

- `pyproject.yaml` lists dependencies author specified
- `uv.lock` lists all packages installed, with specific versions

If lock file exists, *uv* can **reproduce** the exact environment

Common DevOps practice; see also `package.json`, `package-lock.json` for *npm* (*Node.js*); `Cargo.toml`, `Cargo.lock` for *Cargo* (*Rust*)

Also, helps you develop your **Software Bill of Materials (SBOM)**

Build and Run in Docker

Launch a *uv* image to build and run the webserver

```
docker run -it --rm -v $(pwd):/app ghcr.io/astral-sh/uv:python3.14-alpine /bin/sh
```

`-v $(pwd):/app` —Mount current directory as `/app`

`ghcr.io/astral-sh/uv:python3.14-alpine` —Container hosted at ghcr.io

`/bin/sh` —Run this, instead of default command

```
cd /app
```

```
uv run gunicorn web_service:app --bind 0.0.0.0:8000 --timeout 120
```

Activity: Access Server Within Container

1. Find name of running container
2. Inside running container, execute

```
wget -O - http://127.0.0.1:8000/api/visits
```

Should get

```
{"visits": 1}
```

Exposing Ports

Expose port in container to host

```
docker run -it --rm -v $(pwd):/app -p 8000:8000 \
  ghcr.io/astral-sh/uv:python3.14-alpine /bin/sh
```

`-p 8000:8000` — Connect port 8000 on host to port 8000 in container

On host, test with

```
curl http://localhost:8000/api/visits
```


Codespaces Forwards Ports

Codespaces should notice the open port, set up port forwarding

See the *Ports* pane next to *Terminal*

Cleanup

1. Stop running container
2. Delete `.venv` directory that was created

```
rm -rf .venv
```

Dockerfile

Want to automate building and running our service

`Dockerfile` is text file describing how to build a Docker image

Dockerfile

Create file named `Dockerfile`:

```
FROM ghcr.io/astral-sh/uv:python3.14-alpine
```

```
ENV UV_COMPILE_BYTECODE=1
```

```
ENV UV_LINK_MODE=copy
```

```
WORKDIR /app
```

```
COPY . /app
```

```
CMD ["sh", "-c", \  
    "uv run gunicorn web_service:app --bind 0.0.0.0:8000 --timeout 120"]
```

Build Custom Image

Build image with `docker build` command:

```
docker build . -t web-service
```

`.` Build current directory

`-t web-service` Name the image *web-service*

See local Docker images:

```
docker image list
```

Run Custom Image

```
docker run --rm -p 8000:8000 web-service
```

Just specify our local image instead of a remote one

Web service should be available on port 8000

Activity: Visitor Count on Volume

The visitor count has been resetting each time we start a container

The service stores the count in `/var/lib/web-service/visits.txt`

1. Run the container with a volume mounted to host this file
2. See that the count is maintained when you stop and restart the container.

Multi-Stage Builds

Build process often requires tooling not necessary for production

- Here, the *uv* system
- Increases container size
- More components mean more vulnerabilities

Multi-stage builds do build in one image

Then copy relevant files into a second image

Multi-Stage Dockerfile (pt. 1)

```
# Builder stage  
FROM ghcr.io/astral-sh/uv:python3.14-alpine AS builder  
  
ENV UV_COMPILE_BYTECODE=1  
ENV UV_LINK_MODE=copy  
  
WORKDIR /app  
  
COPY . /app  
  
RUN uv sync --frozen --no-dev --no-editable
```

—Continues→

Multi-Stage Dockerfile (pt. 2)

← *Continued* —

```
# Final stage
FROM python:3.14-alpine

WORKDIR /app

COPY --from=builder /app/.venv /app/.venv

ENV PATH="/app/.venv/bin:$PATH"

CMD ["sh", "-c", "gunicorn web_service:app --bind 0.0.0.0:8000 --timeout 120"]
```

Multi-Stage Dockerfile

Build and run as before:

```
docker build . -t web-service-staged
docker run --rm -v $(pwd)/count:/var/lib/web-service -p 8000:8000 \
    web-service-staged
```

Note difference in image size reported by `docker image list`

Security and Reproducibility

Docker image tags are controlled by provider, are mutable

Can, and often are, updated with bugfixes and upgrades

→ `:latest` is always updated!

Lock down version with image **digest**, a SHA-256 hash of image contents

```
docker images --digests
```

Also available on [Docker Hub](https://hub.docker.com/)

Docker Image Digests

Use instead of tag in specifying image

```
docker run python@sha256:c6ead2...cb96fc
```

```
FROM python@sha256:c6ead2...cb96fc
```

```
...
```

A digest specifies one unchanging image ...

... but it can disappear from the source repository.

Environmental Variables

Configuration often accomplished via environmental variables

Defaults can be specified in Dockerfile with `ENV` command:

```
ENV variable=value
```

Defaults overridden when launch container with `-e` argument:

```
docker run -e variable=newvalue ...
```

Configure Service via Variables

← *Continued* —

```
COPY --from=builder /app/.venv /app/.venv

ENV PATH="/app/.venv/bin:$PATH"
ENV PORT=8000
ENV VISIT_COUNTER_FILE="/var/lib/web-service/visits.txt"
ENV OLLAMA_URL="http://localhost:11434"
ENV OLLAMA_MODEL="qwen3:0.6b"

CMD ["sh", "-c", \
    "unicorn web_service:app --bind 0.0.0.0:$PORT --timeout 120"]
```

Change Port via Variable

```
docker build . -t web-service-staged
docker run --rm -v $(pwd)/count:/var/lib/web-service \
  -e PORT=3000 -p 8000:3000 web-service-staged
```

`-e PORT=3000` sets the variable *PORT* to "3000"

`Dockerfile` uses `$PORT` to set the port *unicorn* serves at

`-p 8000:3000` forwards local port 8000 to container port 3000

⇒ Still shows up on port 8000 in Codespaces

Running as User Other than *root*

- Many containers run as container root user
 - Not designed for multi-user operation
- Many container escapes require root access within container
 - Running as non-root user adds an additional hurdle

Best practice for containers running public services: Run service as a non-root user in container

Creating Another User (pt. 1)

```
...  
FROM python:3.14-alpine  
  
ARG USER_UID=1000  
ARG USER_GID=1000  
RUN addgroup -g $USER_GID -S appuser && \  
    adduser -u $USER_UID -S appuser -G appuser  
  
WORKDIR /app  
  
COPY --from=builder --chown=appuser:appuser /app/.venv /app/.venv  
...
```

Creates user *appuser*; switch ownership of copied files to *appuser*

Creating Another User (pt. 2)

```
...  
ENV OLLAMA_MODEL="qwen3:0.6b"  
  
RUN mkdir $(dirname $VISIT_COUNTER_FILE) && \  
    chown appuser:appuser $(dirname $VISIT_COUNTER_FILE)  
  
USER appuser  
  
CMD ["sh", "-c", \  
    "unicorn web_service:app --bind 0.0.0.0:$PORT --timeout 120"]
```

Ensure *appuser* can write to (default) `$VISIT_COUNTER_FILE`

Switch to *appuser* to run the Python service

User IDs and Bind Mounts

- File owner IDs are not translated across the container boundary (?)
 - Not a problem for the root user
 - Can be a problem for non-root users
- Here, we set the *appuser* id to 1000, same as host user owning files
 - **ARG** can be set at **build** time:

```
docker build --build-arg USER_UID=$(id -u) ...
```

- This cannot be adjusted at **run** time

User IDs and Bind Mounts

Other solutions:

- Run docker container as a different user:

```
docker run --user $(id -u):$(id -g) ...
```

- Not baked into image
- Add **entrypoint** that creates new user, *chown*, and run application as new user
- Kubernetes will handle ownership of mounts itself

Container Registries

A **container registry** stores and services container images

- [Docker Hub](#)
- [GitHib Container Registry \(ghcr.io\)](#)
- [Elastic Container Registry](#)

Images uploaded for use on production systems

- Reference as `registry.tld/image:tag` in docker command or file
- Images automatically pulled when needed

Uploading to a Container Registry

1. Log in to the registry:

```
docker login registry.tld
```

2. Tag the local image with the registry name:

```
docker tag local-image registry.tld/image-name:tag
```

3. Push the new tag:

```
docker push registry.tld/image-name:tag
```

N.B. We will use the Gitlab container registry in a later session

Activity: *nginx* Container

nginx is a web server and reverse proxy

- It is distributed in the `nginx` container on Docker Hub
- This container serves files from the `/usr/share/nginx/html` folder
- The HTTP server listens on port 80

Use the `nginx` image to serve the files in `exercises/nginx/html`

See `exercises/nginx/README.md` for these instructions

nginx Solution #1: Mounting a Volume

```
docker run -it --rm -v $(pwd)/html:/usr/share/nginx/html -p 8080:80 nginx
```

This connects port 80 on the container to port 8080 on the host

On some systems, you may not be able to open low ports if not root.

Slightly fancier:

```
docker run -d --rm -v $(pwd)/html:/usr/share/nginx/html:ro -p 8080:80 nginx
```

-d Run as a daemon (disconnect from terminal)

-v ...:ro Mount the volume **read-only**

nginx Solution #2: Custom Image

Create a Dockerfile:

```
FROM nginx:latest  
  
COPY html /usr/share/nginx/html
```

Build an image: `docker build . -t my-nginx`

Run a container: `docker run --rm -p 8080:80 my-nginx`

Activity: *ollama* Inference

ollama is an **inference server** for large language models (LLMs)

We will use it to serve the *qwen3:0.6b* model

It is packaged in the `ollama/ollama` image on [Docker Hub](#)

Once a container is running, the model needs to be download.
Execute within the running container:

```
ollama pull qwen3:0.6b
```

Activity: *ollama* Inference (Cont.)

The inference server runs on port 11434

You can get a response from the model with the following command:

```
curl http://localhost:11434/api/generate -d '{  
  "model": "qwen3:0.6b",  
  "prompt": "What is 2+2?",  
  "stream": false  
'
```

These instructions are available in [exercises/ollama/README.md](#)

ollama Solution

```
docker run -d -p 11434:11434 --name ollama-container ollama/ollama  
docker exec ollama-container ollama pull qwen3:0.6b
```

Then access port 11343 on localhost

Feedback



form.typeform.com/to/MNqDLRtz